# Roadmap Catalina Muñoz

**Machine learning for combinatorial optimization**

Dr. Franco Quezada

Assistant Professor
Universidad de Santiago de Chile

franco.quezada@usach.cl

## Executive summary

The current manuscript has a publishable core, but it should be repositioned more carefully. The paper should not be framed as a broad solution to "machine learning for combinatorial optimization." Instead, it should be presented as a **small-data, feasibility-aware matheuristic for repeated operational 0-1 MILPs**. The central idea is strong: learn binary decisions from solved instances, project the prediction into a feasible binary vector using a Phase-I-inspired rule, and solve the resulting restricted MILP.

The next stage should not start by adding a much more complex learning method. The priority is to prove, through a clean **ablation study**, which components are responsible for the observed performance. After that, the team can decide whether to add a second 0-1 MILP class and whether to include a regret-aware extension.

**Recommended order:**

1. Reposition the paper and narrow the claims.
2. Reproduce the current MCLSP results with clean scripts.
3. Build the ablation table and report feasibility, gap, runtime, and number of flips.
4. Add robustness tests across random seeds, training sizes, and instance difficulty.
5. Add a minimal regret-aware experiment: select models by validation objective regret, not only by classification accuracy.
6. Only then decide whether to add a second 0-1 MILP problem class.

# Índice


1 How to reposition the work 3
1.1 Current risk . . . 3
1.2 Recommended positioning . . . 3
1.3 Possible titles . . . 3
1.4 Reformulated contribution statement . . . 3

2 Research questions to answer 5

3 General roadmap diagram 6

4 Ablation table: purpose and structure 7
4.1 Canonical ablation variants . . . 7
4.2 Main ablation table template . . . 8
4.3 Example full table . . . 8

5 Experiments needed to build the ablation table 9
5.1 Experiment 0: reproducibility and clean pipeline . . . 9
5.2 Experiment 1: baseline comparison . . . 9
5.3 Experiment 2: feasibility rule ablation . . . 9
5.4 Experiment 3: feature-set ablation . . . 10
5.5 Experiment 4: post-processing ablation . . . 10
5.6 Experiment 5: robustness across seeds and splits . . . 11
5.7 Experiment 6: training-set size sensitivity . . . 11
5.8 Experiment 7: instance difficulty and scalability . . . 11

6 Should we add another 0-1 MILP problem? 12
6.1 Path A: conservative and faster . . . 12
6.2 Path B: broader and stronger . . . 12

7 How to compare with the literature 13
7.1 Computational baselines . . . 13
7.2 Literature positioning . . . 13

8 Regret-aware extension 14
8.1 Core idea . . . 14
8.2 Minimum viable regret experiment . . . 14
8.3 Second-level regret-aware variant: weighted classification . . . 14
8.4 Third-level variant: regret-aware feasibility rule . . . 15

9 Suggested timeline 16

Referencias 17

# 1. How to reposition the work

## 1.1. Current risk

The manuscript currently studies a predict-then-optimize framework for 0-1 MILPs and tests it on the Multi-item Capacitated Lot-Sizing Problem (MCLSP). It uses classical machine learning models, feature sets based on model parameters and linear relaxations, and a feasibility-aware discrimination rule inspired by Phase I of the simplex method. In the strongest setting, the reported results show very large runtime reductions and small optimality gaps.

The risk is that reviewers may interpret the paper as making a broad claim about machine learning for combinatorial optimization, while the evidence is mostly based on one problem class and one strong feature family. A reviewer may ask:

- Is the improvement due to machine learning, or mostly due to the strengthened linear relaxation?
- Is the feasibility-aware rule essential, or would simple rounding and repair give similar results?
- Does the method generalize beyond MCLSP?

## 1.2. Recommended positioning

The paper should be framed as follows:

> *We study whether a feasibility-aware projection of ML-predicted binary decisions can produce reliable small-data matheuristics for repeated operational 0-1 MILPs.*

This positioning is stronger because it emphasizes what the paper actually does well:

- It uses small training sets.
- It preserves feasibility before solving the restricted problem.
- It is useful for repeated operational decision-making.
- It evaluates runtime-quality tradeoffs rather than pure prediction accuracy.

## 1.3. Possible titles

- *A feasibility-aware small-data matheuristic for repeated 0-1 operational MILPs*
- *A feasibility-aware predict-then-optimize matheuristic for operational MILPs*
- If the paper remains MCLSP-only: *A feasibility-aware predict-then-optimize matheuristic for capacitated lot-sizing*

## 1.4. Reformulated contribution statement

The contribution should be written in a controlled way:

1. We propose a feasibility-aware discrimination rule that transforms continuous ML scores into feasible binary decisions through a single auxiliary optimization problem.

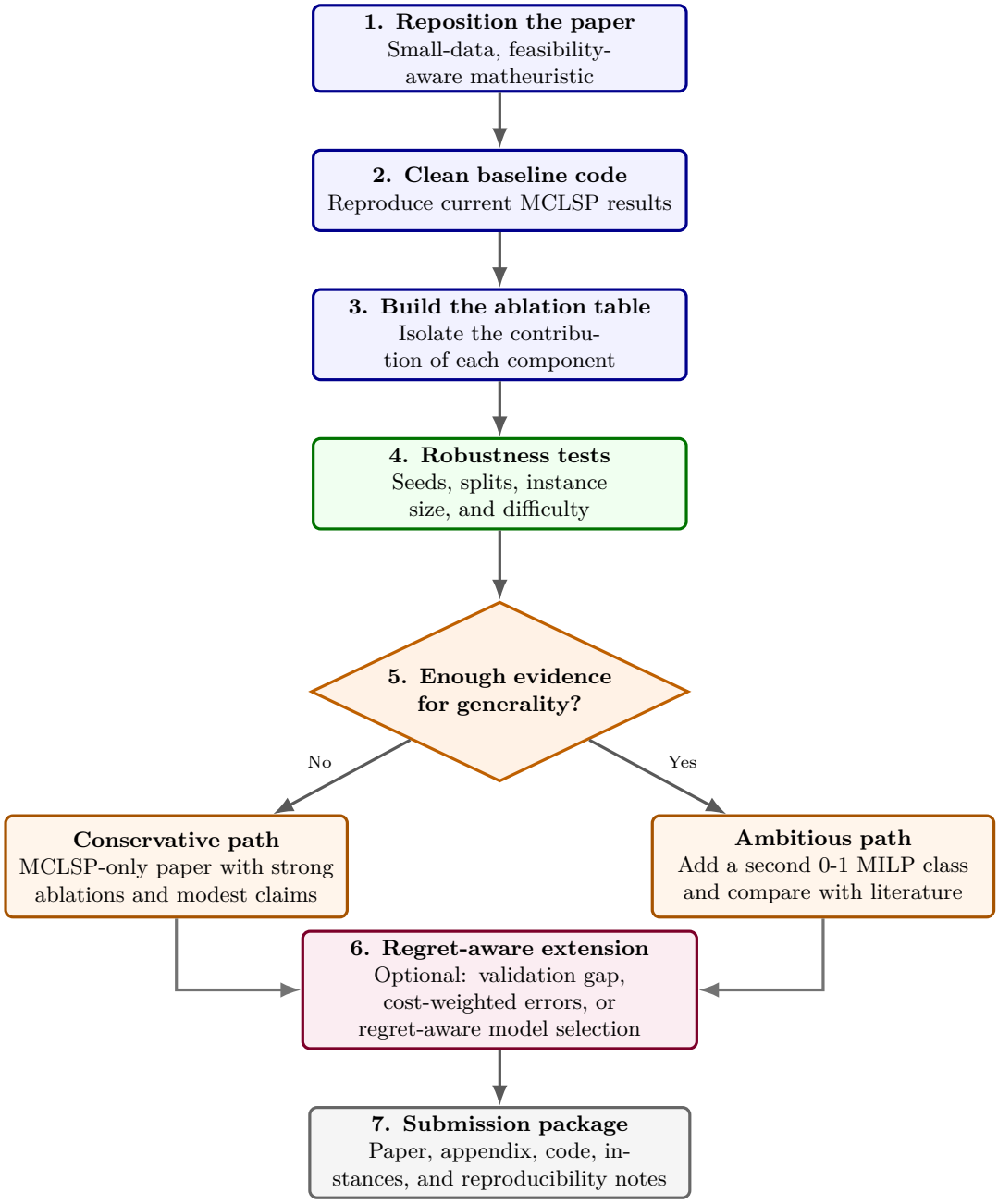
2. We integrate this rule into a predict-then-optimize matheuristic for repeated 0-1 operational MILPs.
3. We quantify the role of different feature families, including original model parameters, basic linear relaxation features, and strengthened relaxation features.
4. We perform an ablation study to isolate the effect of ML prediction, relaxation features, feasibility recovery, and post-processing.
5. We evaluate scalability and robustness under changes in problem size and capacity tightness.

## 2. Research questions to answer

The student should organize the experimental work around explicit research questions. Each question should correspond to one table or figure.

| Code | Research question | Evidence needed |
|---|---|---|
| RQ1 | Does ML add value beyond simple LP-based decisions? | Compare ML-based variants against LP rounding and no-ML relaxation baselines. |
| RQ2 | Does the feasibility-aware rule matter? | Compare threshold-only prediction with the feasibility-aware discrimination rule. |
| RQ3 | Is feature set C doing most of the work? | Compare feature sets A, B, and C under the same ML model and same repair rule. |
| RQ4 | Is post-processing important? | Compare solutions before and after removing unnecessary setup activations. |
| RQ5 | Is the method stable under small-data training? | Repeat experiments across random seeds, splits, and training-set sizes. |
| RQ6 | When does the method scale? | Test larger item sets, different planning horizons, and tighter capacity regimes. |
| RQ7 | Can the framework be claimed as general? | Add a second 0-1 MILP class, or narrow the paper to MCLSP. |
| RQ8 | Does cost-sensitive or regret-aware learning improve decision quality? | Compare accuracy-based selection with regret-based validation selection and cost-weighted variants. |

## 3. General roadmap diagram



# 4. Ablation table: purpose and structure

An ablation table removes or replaces one component of the method at a time. The goal is to show not only that the full pipeline works, but **why** it works.

The full method contains several components:

1. A source of prediction scores $z$ for the binary variables.
2. A feature set: A, B, or C.
3. A classification threshold.
4. The feasibility-aware discrimination rule.
5. A restricted MILP obtained by fixing the predicted binary vector.
6. A post-processing step that removes unnecessary setup activations.

The ablation table should compare variants that isolate each component.

## 4.1. Canonical ablation variants

| **ID** | **Variant** | **Prediction source** | **Feasibility rule?** | **Question answered** |
|---|---|---|---|---|
| M0 | Solver baseline | No prediction; full MILP solved by Gurobi | No | What is the reference solution and runtime? |
| M1 | Threshold-only ML | ML scores $z$ with $z \geq \beta$ | No | How often are raw ML predictions feasible? |
| M2 | Threshold-only + restricted MILP | ML thresholded vector fixed in MILP when feasible | No | What is the quality without repair? |
| M3 | LP rounding | Basic LP relaxation values rounded | No | Does a simple no-ML method already work? |
| M4 | LP rounding + feasibility rule | LP relaxation values used as scores | Yes | What does the feasibility rule do without ML? |
| M5 | ML-A + feasibility rule | ML trained with original model features | Yes | How much can original parameters explain? |
| M6 | ML-B + feasibility rule | ML trained with basic LP relaxation features | Yes | Does the basic relaxation help? |
| M7 | ML-C + feasibility rule | ML trained with strengthened relaxation features | Yes | Is feature set C the main driver? |
| M8 | Full method | ML-C + feasibility rule + post-processing | Yes | What is the final proposed pipeline? |

| | | | | |
|---|---|---|---|---|
| M9 | Regret-aware selection | Model selected by validation objective gap | Yes | Does optimization-aware selection beat accuracy-based selection? |

## 4.2. Main ablation table template

The final paper should include one compact table like this for each major instance group. The detailed version can go in the appendix.

| **Variant** | **Features** | **Feas. (%)** | **Flips** | **Repair** | **Obj. gap** | **Time gap** | **Solved** |
|---|---|---|---|---|---|---|---|
| M0 Solver | – | 100 | – | – | 0.00 | 0.00 | – |
| M1 Threshold ML | A/B/C | | 0 | 0 | | | |
| M3 LP rounding | B/C | | 0 | 0 | | | |
| M4 LP + rule | B/C | | | | | | |
| M5 ML-A + rule | A | | | | | | |
| M6 ML-B + rule | B | | | | | | |
| M7 ML-C + rule | C | | | | | | |
| M8 Full method | C | 100 | | | | | |
| M9 Regret-aware | C | 100 | | | | | |

**Important:** The table should include all computational time used by the method: feature generation, ML prediction, feasibility rule solution, restricted MILP solution, and post-processing.

## 4.3. Example full table

Cuadro 4: Ablation table for the proposed feasibility-aware predict-then-optimize framework.

| **Variant** | **ML model** | **Features** | **Feasibility-aware rule** | **Post-processing** | **Feasible solutions (%)** | **Mean Gap Obj (%)** | **Mean Gap Time (%)** | **Mean flips** |
|---|---|---|---|---|---|---|---|---|
| Solver baseline | — | — | — | — | 100 | 0.00 | 0.00 | — |
| Threshold only | RF | C | No | No | 72 | 0.45 | -98.9 | 0 |
| Threshold + post-processing | RF | C | No | Yes | 72 | 0.39 | -98.8 | 0 |
| Feasibility rule only | No ML | LP scores | Yes | Yes | 100 | 1.20 | -98.5 | 18.4 |
| ML + basic LP | RF | B | Yes | Yes | 100 | 2.13 | -99.7 | 9.1 |
| ML + strengthened LP | RF | C | Yes | Yes | 100 | 0.67 | -99.7 | 2.8 |
| Full proposed method | RF | C | Yes | Yes | 100 | 0.67 | -99.7 | 2.8 |

# 5. Experiments needed to build the ablation table

## 5.1. Experiment 0: reproducibility and clean pipeline

**Objective.** Ensure that the current results can be reproduced from clean scripts.
**Tasks.**

1. Create a single configuration file defining instance size, capacity ratio, random seed, feature set, ML model, solver time limit, and output folder.
2. Save all generated instances with unique identifiers.
3. Save the optimal or best-known solution for every instance.
4. Save feature-generation time, training time, prediction time, repair time, restricted MILP time, and total time separately.
5. Reproduce the current tables for feature sets A, B, and C.

**Output.** A reproducibility table confirming that the current manuscript results can be regenerated.

## 5.2. Experiment 1: baseline comparison

**Objective.** Establish fair reference points.
**Methods to compare.**

- Full Gurobi solve with the same time limit used in the manuscript.
- Threshold-only ML prediction.
- LP rounding using feature set B or C values as scores.
- LP rounding plus feasibility-aware rule.
- Gurobi warm start from the predicted solution, if easy to implement.
- Full proposed method.

**Output.** A table showing feasibility rate, objective gap, runtime, and number of solved instances.

## 5.3. Experiment 2: feasibility rule ablation

**Objective.** Prove that the Phase-I-inspired rule is necessary and useful.
**Procedure.**

1. Generate raw ML predictions using $z \geq \beta$.
2. Check whether the restricted MILP with this raw binary vector is feasible.
3. Apply the feasibility-aware rule.
4. Record how many variables are flipped.
5. Solve the restricted MILP after repair.

6. Compare objective value and runtime before and after repair.

**Metrics.**

- Raw feasibility rate.
- Repaired feasibility rate.
- Mean and maximum number of flips.
- Repair time.
- Objective gap after repair.
- Correlation between number of flips and objective gap.

## 5.4. Experiment 3: feature-set ablation

**Objective.** Quantify the role of feature sets A, B, and C.
**Procedure.**

1. Fix the ML model, preferably RF if it remains the best model.
2. Train the model separately with feature sets A, B, and C.
3. Apply the same threshold, same repair rule, and same restricted MILP procedure.
4. Compare accuracy, feasibility, objective gap, runtime, and flips.

**Interpretation.** If feature set C dominates, the paper should not hide this. Instead, the contribution becomes: strengthened relaxations generate informative signals, and the feasibility-aware rule turns those signals into reliable feasible decisions.

## 5.5. Experiment 4: post-processing ablation

**Objective.** Measure whether removing unnecessary setup activations improves the objective.
**Procedure.**

1. Solve the restricted MILP after the feasibility-aware rule.
2. Identify binary setup variables fixed to one but associated with zero production.
3. Remove these unnecessary setups.
4. Recompute objective value.

**Metrics.**

- Number of removed setup variables.
- Objective improvement from post-processing.
- Frequency with which post-processing changes the solution.

## 5.6. Experiment 5: robustness across seeds and splits

**Objective.** Show that the method is not an artifact of one split.
**Procedure.**

1. Use at least 10 random seeds.
2. Ensure splits are made at the instance level, not at the row or variable level.
3. Report mean and standard deviation for all key metrics.
4. If possible, report 95 % confidence intervals.

**Output.** A robustness table with mean ± standard deviation for GapObj, GapTime, feasibility rate, and flips.

## 5.7. Experiment 6: training-set size sensitivity

**Objective.** Support the small-data claim.
**Procedure.**

1. Train with different numbers of solved instances, for example $N \in \{6, 9, 12, 18, 24\}$ if computationally feasible.
2. Keep the test set fixed.
3. Evaluate how solution quality and feasibility evolve as training data increases.

**Expected insight.** If performance is already strong with few instances, this reinforces the main story of the paper.

## 5.8. Experiment 7: instance difficulty and scalability

**Objective.** Identify when the method is most valuable.
**Factors to vary.**

- Number of items.
- Planning horizon.
- Capacity tightness.
- Setup-to-holding cost ratio.

**Output.** A table or heatmap showing where the proposed method has the best runtime-quality tradeoff.

# 6. Should we add another 0-1 MILP problem?

There are two possible publication paths.

## 6.1. Path A: conservative and faster

Keep the paper focused on MCLSP. In this case, the title and claims must be narrowed. The paper becomes a strong, problem-specific matheuristic paper.

**Use this path if:**

- The ablation study is strong.
- Time is limited.
- Adding a second problem would delay the paper too much.

## 6.2. Path B: broader and stronger

Add a second 0-1 MILP family. This supports the claim that the feasibility-aware framework is not only a lot-sizing trick.

**Recommended candidates.**

| **Problem** | **Why it fits** | **Risk** |
|---|---|---|
| Capacitated facility location | Has binary opening decisions and continuous assignment/flow decisions. Similar fixed-charge structure but different from MCLSP. | Need instance generator and fair baselines. |
| Fixed-charge transportation | Very close to the general MILP structure with activation variables. | May be perceived as too similar to MCLSP if not designed carefully. |
| p-median or facility location variants | Classical, interpretable, and easy to explain. | Some variants are mostly binary and may not test the continuous-restricted MILP logic. |
| Generalized assignment | Challenging and widely studied. | Feasibility rule may need more adaptation. |
| Set covering with activation variables | Simple and scalable. | Less aligned with the current continuous-plus-binary framework. |

**Recommendation.** First complete the MCLSP ablation study. Then add a second problem only if the ablation confirms that the method has an independent contribution beyond feature set C. The best candidate is capacitated facility location or fixed-charge transportation.

# 7. How to compare with the literature

The paper should include two types of comparisons.

## 7.1. Computational baselines

These are methods to implement and run:

- Full MILP solver with time limit.
- LP rounding.
- LP rounding plus feasibility-aware rule.
- ML thresholding without repair.
- ML with feasibility-aware repair.
- Optional: solver warm start from predicted variables.

## 7.2. Literature positioning

These methods do not all need to be reimplemented, but they should be discussed clearly:

- Predict-then-optimize and decision-focused learning methods, especially SPO and SPO+.
- Neural diving and neural branching approaches for MIP.
- Learning-based variable fixing and correction methods.
- Lot-sizing ML approaches, especially LSTM or transformer-based methods for setup prediction.

The comparison should be honest: the proposed paper is not trying to beat all neural methods on all benchmarks. Its niche is **small-data, interpretable, feasibility-aware, operational MILP matheuristics**.

# 8. Regret-aware extension

The current ML models are trained to reduce prediction error in the binary vector. However, not all binary mistakes have the same downstream cost. In MCLSP, predicting a setup variable incorrectly can have very different consequences depending on item, period, demand, capacity, and cost structure. A false negative may make the restricted problem infeasible. A false positive may only add an unnecessary setup, and it may later be removed by post-processing. Therefore, standard accuracy is not fully aligned with optimization performance.

## 8.1. Core idea

The regret-aware extension asks:

> *Can we train or select ML models by the cost of the optimization error they induce, rather than by the number of binary variables they classify correctly?*

A natural instance-level regret measure is:

$$_i(\hat{y}) = \frac{v_i(\hat{y}) - v_i(y_i^*)}{\max\{1, |v_i(y_i^*)|\}},$$

where $v_i(\hat{y})$ is the objective value obtained after applying the feasibility rule and solving the restricted MILP, and $v_i(y_i^*)$ is the optimal or best-known objective value.

## 8.2. Minimum viable regret experiment

The easiest useful version is not to change the training algorithm yet. Instead, change the **model selection criterion**.

1. Train the same candidate ML models and hyperparameters as before.
2. On the validation fold, apply the complete optimization pipeline: prediction, feasibility rule, restricted MILP, post-processing.
3. Select the model that minimizes validation GapObj or validation regret.
4. Compare this against the current accuracy/CV-based selection.

This directly tests whether a model with lower classification accuracy can produce better optimization decisions.

## 8.3. Second-level regret-aware variant: weighted classification

After the minimum experiment, we can try cost-weighted training. Instead of treating all binary variables equally, assign a weight $w_{ip}$ to each binary label $y_{ip}$, where $w_{ip}$ estimates the cost of misclassifying variable $p$ in instance $i$.

Possible weights:

- Exact counterfactual weight: solve the model after forcing one variable to the wrong value and measure objective degradation. This is accurate but expensive.
- Approximate weight from objective coefficients, setup costs, demand, capacity tightness, or reduced costs.

- Validation-based weight from observed damage caused by flips or false negatives.

Then train using weighted classification, for example through sample weights in random forests or weighted logistic loss for other classifiers.

## 8.4. Third-level variant: regret-aware feasibility rule

The feasibility rule can also be modified. Instead of minimizing only the number of flips and confidence-weighted penalties, each flip can receive a predicted cost:

$$\min \sum_p w_p^{0\to 1} q_p^{0\to 1} + \sum_p w_p^{1\to 0} q_p^{1\to 0},$$

where $q_p^{0\to 1}$ and $q_p^{1\to 0}$ indicate whether the predicted binary value is flipped. The weights combine ML confidence and estimated optimization damage.

This is attractive because it keeps the feasibility-aware structure of the current paper but makes the correction rule more optimization-aware.

## 9. Suggested timeline

| **Weeks** | **Main task** | **Deliverable** |
|---|---|---|
| 1–2 | Reproduce current results and clean the codebase | Reproducibility report and experiment configuration files. |
| 3–4 | Implement baselines and ablation variants | First ablation table for MCLSP. |
| 5–6 | Run robustness, seed, and training-size experiments | Robustness tables and plots. |
| 7 | Add minimum viable regret-aware selection | Accuracy-selected vs regret-selected comparison. |
| 8–9 | Decide whether to add a second 0-1 MILP | Either second-problem results or narrowed MCLSP paper. |
| 10–18 | Rewrite paper and prepare appendix | Submission-ready manuscript, appendix, and code release plan. |

## Referencias


[1] Cornejo, J., Davila-Galvez, S., Quezada, F., and Rojo-Gonzalez, L. (2026). *Feasibility-aware predict-then-optimize framework for 0-1 mixed-integer linear problems*. Working manuscript.

[2] Bengio, Y., Lodi, A., and Prouvost, A. (2021). Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2), 405–421.

[3] Elmachtoub, A. N., and Grigas, P. (2022). Smart "predict, then optimize". *Management Science*, 68(1), 9–26.

[4] Vanderschueren, T., Verdonck, T., Baesens, B., and Verbeke, W. (2022). Predict-then-optimize or predict-and-optimize? An empirical evaluation of cost-sensitive learning strategies. *Information Sciences*, 594, 400–415.

[5] Nair, V., et al. (2020). Solving mixed integer programs using neural networks. arXiv:2012.13349.

[6] Yilmaz, D., and Buyuktahtakin, I. E. (2024). An expandable machine learning-optimization framework to sequential decision-making. *European Journal of Operational Research*, 314(1), 280–296.

[7] Liu, H., Wang, J., Geng, Z., Li, X., Zong, Y., Zhu, F., Hao, J., and Wu, F. (2025). Apollo-MILP: An alternating prediction-correction neural solving framework for mixed-integer linear programming. arXiv:2503.01129.